

for NLI tasks. The Facebook NLP team used the NLI task as the transfer learning task to generate sentence embeddings that can be used for any general-purpose downstream task. Their encoder consisted of bi-directional LSTM followed by a max-pooling layer (one can also use mean pooling instead of a max-pooling layer) for obtaining the fixed length sentence-embedding vector. The team showed that its sentence-embedding model performed better than the SkipThought sentence-embedding model on a set of downstream NLP tasks, which included multi-class text classification, entailment and paraphrase detection, caption image retrieval, etc. In fact, they created an evaluation tool for universal sentence-embeddings to evaluate these tasks with any general-purpose sentence embeddings. Both Inference and SentEval are publicly available. Hence I would encourage our readers to experiment with these.

While sentence-embeddings trained with the SNLI corpus showed reasonable performance on downstream NLP tasks, there have been other efforts in identifying supervised training tasks which can transfer well to generalised NLP downstream tasks. Instead of training on one task to obtain generalised sentence-embeddings, the focus shifted to a multi-task learning approach to learn universal sentence-embeddings.

The key idea behind using multi-task learning is that general-purpose sentence-embeddings need to capture various different aspects of a sentence for them to be useful for multiple downstream tasks. Each of the different tasks in the multi-task learning (MTL) framework enables the sentence encoder to learn a different aspect. For instance, a constituency parsing task may enable the sentence-embedding encoder to learn about syntax, whereas the natural language inference task may enable the encoder to learn the aspects of semantics. Hence the idea is to learn the general-purpose sentence-embeddings in the context of a simple one-to-many multi-task learning framework, wherein the same encoder is shared across multiple weakly related tasks.

The encoder is a simple recurrent neural network model and is trained by switching between different tasks. This enables the learnt sentence-embedding to encode the inductive biases of different tasks. This approach was first proposed in the ICLR 2018 paper by the Canada based educational institute MILA and the Microsoft Montreal group. This was followed by the ‘universal sentence encoder’ work from Google.

The Universal Sentence Encoder from Google is publicly available and can be used as a pre-trained model for a number of downstream tasks. In fact, the model is readily available in the TensorFlow hub and can easily be integrated into your application with the following few lines of code:

```
modurl = https://tfhub.dev/google/universal-sentence-encoder/2
embed = hub.Module(modurl)
embeddings = embed([
    "This article covers the various sentence embeddings",
    "I have a sentence for which I would like to get its
    embedding"])
print session.run(embeddings)
```

Now you have the sentence-embeddings for the two sentences in your corpus in the above example snippet, and you can use them as input to any of your downstream tasks – be it text classification, text similarity or paraphrase detection.

Google’s Universal Sentence Encoder supports two types of encoders. The first is the Deep Averaging Network (DAN) encoder, and the second is a multi-head attention transformer based encoder. Both these encoders are first trained with unsupervised training data from various online sources such as Wikipedia, Google News, question-answer forums, etc. They are then further trained on the SNLI corpus, which is a human-labelled corpus. The output of the encoder is a 512 dimensional sentence embedding. The DAN based model is lightweight compared to the transformer based encoder, but the transformer based encoder model is more accurate. However, memory usage for the DAN based model is likely to be larger compared to that of the transformer based model. So based on your downstream task requirements, you can choose to use either of the two universal sentence encoding models.

So far, we have discussed a number of universal sentence-embeddings. Given these universal sentence-embedding models, I would like to pose a question to our readers. Let us assume that you are given an unlabelled text corpus consisting of a set of documents. You need to cluster these documents into separate clusters. One classical way of approaching this problem is to construct a document representation vector or feature representation vector for each of these documents, and then use a standard clustering algorithm such as k-means. The document representation vector can be as simple as a TF-IDF representation of word frequencies of the corpus vocabulary. We can also consider other document representations such as Doc2vec or build a neural representation using auto-encoders, which can then be fed to a clustering algorithm. Another approach is to use some of these general-purpose sentence-embedding methods to represent each document and then run a clustering algorithm on the document representation. It would be interesting to see how these different document representations impact the clustering performance and which of these representations are clustering-task friendly.

We can also use the sentence-embedding representation as an input to the text classification problem we have been discussing in the last two columns, namely, classifying insincere questions on Quora. I would encourage our readers to try this and let me know whether it improves performance.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month. 

 By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing.